#2

```
# ========================================================
# PRACTICAL 2: EDA AUTOMATION PROJECT
# ========================================================

# STEP 1: IMPORT REQUIRED LIBRARIES

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# STEP 2: CREATE SAMPLE DATASET

data = {
    'Name': ['Amit', 'Rahul', 'Priya', 'Sneha',
             'Neha', 'Raj', 'Anjali', 'Karan'],
    'Age': [20, 21, 20, 22, 21, 20, 22, 21],
    'Marks': [85, 75, 95, 65, 80, 70, 90, 60],
    'Attendance': [90, 80, 95, 70, 85, 75, 92, 65]
}

df = pd.DataFrame(data)

# STEP 3: DISPLAY DATASET

print("Original Dataset:")
print(df)

# STEP 4: DISPLAY FIRST FIVE RECORDS

print("\nFirst Five Records:")
print(df.head())

# STEP 5: DISPLAY DATASET INFORMATION

print("\nDataset Information:")
df.info()

# STEP 6: GENERATE STATISTICAL SUMMARY

print("\nStatistical Summary:")
print(df.describe())

# STEP 7: CHECK MISSING VALUES

print("\nMissing Values:")
print(df.isnull().sum())

# STEP 8: CHECK DUPLICATE RECORDS

print("\nDuplicate Records:")
print(df.duplicated().sum())

# STEP 9: CALCULATE AVERAGE MARKS

average_marks = df['Marks'].mean()
print("\nAverage Marks:", average_marks)

# STEP 10: FIND HIGHEST AND LOWEST MARKS

print("Highest Marks:", df['Marks'].max())
print("Lowest Marks:", df['Marks'].min())

# STEP 11: VISUALIZE STUDENT MARKS

plt.figure(figsize=(8, 5))
plt.bar(df['Name'], df['Marks'])

plt.xlabel("Student Name")
plt.ylabel("Marks")
plt.title("Student Marks Analysis")
plt.tight_layout()
plt.show()

# STEP 12: CREATE CORRELATION HEATMAP

plt.figure(figsize=(6, 4))

sns.heatmap(
    df[['Age', 'Marks', 'Attendance']].corr(),
    annot=True,
    cmap='coolwarm'
)

plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()

# STEP 13: CREATE MARKS HISTOGRAM

plt.figure(figsize=(6, 4))

plt.hist(df['Marks'], bins=5, edgecolor='black')

plt.xlabel("Marks")
plt.ylabel("Number of Students")
plt.title("Distribution of Student Marks")
plt.tight_layout()
plt.show()

# STEP 14: DISPLAY FINAL RESULT

print("\nEDA Completed Successfully!")


pip install pandas numpy matplotlib seaborn

python eda_project.py
```

#4

```
# ========================================================
# PRACTICAL 4: INTERPRETABLE AI
# Enhancing Model Transparency
# ========================================================

# STEP 1: INSTALL REQUIRED LIBRARIES
# Run this command in terminal:
# pip install pandas numpy matplotlib seaborn scikit-learn

# STEP 2: IMPORT LIBRARIES

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report

# STEP 3: LOAD SAMPLE DATASET

iris = load_iris()

df = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

df['Target'] = iris.target

print("Sample Dataset:")
print(df.head())

# STEP 4: SEPARATE INPUT AND OUTPUT

X = df.drop('Target', axis=1)
y = df['Target']

# STEP 5: SPLIT DATASET INTO TRAINING AND TESTING

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# STEP 6: CREATE INTERPRETABLE MODEL

model = DecisionTreeClassifier(
    max_depth=3,
    random_state=42
)

model.fit(X_train, y_train)

# STEP 7: MAKE PREDICTIONS

y_pred = model.predict(X_test)

print("\nModel Predictions:")
print(y_pred)

# STEP 8: CHECK MODEL ACCURACY

accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:", accuracy)

# STEP 9: DISPLAY CLASSIFICATION REPORT

print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))

# STEP 10: DISPLAY FEATURE IMPORTANCE

importance = model.feature_importances_

feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': importance
})

print("\nFeature Importance:")
print(feature_importance.sort_values(
    by='Importance',
    ascending=False
))

# STEP 11: VISUALIZE FEATURE IMPORTANCE

plt.figure(figsize=(8, 5))

sns.barplot(
    data=feature_importance,
    x='Importance',
    y='Feature'
)

plt.title("Feature Importance")
plt.xlabel("Importance Score")
plt.ylabel("Features")

plt.show()

# STEP 12: VISUALIZE DECISION TREE
```

```
plt.figure(figsize=(15, 8))

plot_tree(
    model,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True,
    rounded=True
)

plt.title("Interpretable Decision Tree")

plt.show()

# STEP 13: EXPLAIN ONE PREDICTION

sample = X_test.iloc[[0]]

prediction = model.predict(sample)

print("\nSample Input:")
print(sample)

print("\nPredicted Flower:",
      iris.target_names[prediction[0]])

print("\nActual Flower:",
      iris.target_names[y_test.iloc[0]])

print("\nPractical Completed Successfully!")


pip install pandas numpy matplotlib seaborn scikit-learn

python interpretable_ai.py
```

#6

```
# ============================================================
# PRACTICAL 6: END-TO-END ML PIPELINE USING MLFLOW
# ============================================================

# STEP 1: IMPORT REQUIRED LIBRARIES

import pandas as pd
import numpy as np
import mlflow
import mlflow.sklearn

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# STEP 2: LOAD SAMPLE DATASET

iris = load_iris()

X = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

y = iris.target

print("Dataset Loaded Successfully")
print(X.head())

# STEP 3: SPLIT DATASET

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("\nTraining Data:", X_train.shape)
print("Testing Data:", X_test.shape)

# STEP 4: SET MLFLOW EXPERIMENT

mlflow.set_experiment("Iris_ML_Pipeline")

# STEP 5: START MLFLOW RUN

with mlflow.start_run():

    # STEP 6: CREATE MACHINE LEARNING MODEL

    model = RandomForestClassifier(
        n_estimators=100,
        max_depth=3,
        random_state=42
    )

    # STEP 7: TRAIN MODEL

    model.fit(X_train, y_train)

    print("\nModel Training Completed")

    # STEP 8: MAKE PREDICTIONS

    predictions = model.predict(X_test)

    print("\nPredictions:")
    print(predictions)

    # STEP 9: EVALUATE MODEL

    accuracy = accuracy_score(
```

```
        y_test,
        predictions
    )

    print("\nModel Accuracy:", accuracy)

    # STEP 10: LOG PARAMETERS IN MLFLOW

    mlflow.log_param("n_estimators", 100)
    mlflow.log_param("max_depth", 3)
    mlflow.log_param("algorithm", "Random Forest")

    # STEP 11: LOG METRICS

    mlflow.log_metric("accuracy", accuracy)

    # STEP 12: SAVE MODEL IN MLFLOW

    mlflow.sklearn.log_model(
        model,
        name="iris_random_forest"
    )

    print("\nParameters Logged Successfully")
    print("Metrics Logged Successfully")
    print("Model Saved Successfully")

# STEP 13: DISPLAY FINAL RESULT

print("\nMLflow Pipeline Completed Successfully!")

print("Open MLflow Dashboard using:")
print("mlflow ui --port 5000")


pip install pandas numpy scikit-learn mlflow
python -m pip install mlflow
python -m pip show mlflow
python -m mlflow ui --port 5000



python mlflow_project.py
```

#8

```
# ============================================================
# PRACTICAL 8: BIG DATA EXPLORATION USING DASK
# ============================================================

# STEP 1: IMPORT REQUIRED LIBRARIES

import dask
import dask.dataframe as dd
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# STEP 2: CREATE SAMPLE DATASET

data = {
    'Name': ['Amit', 'Rahul', 'Priya', 'Sneha',
             'Neha', 'Raj', 'Anjali', 'Karan'],

    'Age': [20, 21, 20, 22, 21, 20, 22, 21],

    'Marks': [85, 75, 95, 65, 80, 70, 90, 60],

    'Attendance': [90, 80, 95, 70, 85, 75, 92, 65]
}

df = pd.DataFrame(data)

print("Original Dataset:")
print(df)

# STEP 3: CONVERT PANDAS DATAFRAME TO DASK DATAFRAME

ddf = dd.from_pandas(df, npartitions=2)

print("\nDask DataFrame Created Successfully")

# STEP 4: DISPLAY FIRST FIVE RECORDS

print("\nFirst Five Records:")
print(ddf.head())

# STEP 5: DISPLAY DATASET INFORMATION

print("\nDataset Information:")
print(ddf.dtypes)

# STEP 6: CALCULATE TOTAL NUMBER OF RECORDS

total_records = ddf.shape[0].compute()

print("\nTotal Records:", total_records)

# STEP 7: CALCULATE AVERAGE MARKS

average_marks = ddf['Marks'].mean().compute()

print("\nAverage Marks:", average_marks)

# STEP 8: FIND MAXIMUM AND MINIMUM MARKS

max_marks = ddf['Marks'].max().compute()

min_marks = ddf['Marks'].min().compute()

print("\nMaximum Marks:", max_marks)
print("Minimum Marks:", min_marks)

# STEP 9: CALCULATE AVERAGE ATTENDANCE
```

```
average_attendance = ddf['Attendance'].mean().compute()

print("\nAverage Attendance:", average_attendance)

# STEP 10: FILTER STUDENTS WITH MARKS ABOVE 75

high_marks = ddf[ddf['Marks'] > 75].compute()

print("\nStudents Scoring Above 75:")
print(high_marks)

# STEP 11: GROUP DATA BY AGE

age_group = ddf.groupby('Age')['Marks'].mean().compute()

print("\nAverage Marks by Age:")
print(age_group)

# STEP 12: SORT STUDENTS BY MARKS

sorted_data = ddf.sort_values(
    'Marks',
    ascending=False
).compute()

print("\nStudents Sorted by Marks:")
print(sorted_data)

# STEP 13: VISUALIZE MARKS USING BAR CHART

plt.figure(figsize=(8, 5))

plt.bar(df['Name'], df['Marks'])

plt.xlabel("Student Name")
plt.ylabel("Marks")
plt.title("Student Marks Analysis Using Dask")

plt.show()

# STEP 14: DISPLAY FINAL RESULT

print("\nBig Data Analysis Completed Successfully!")
```

```
pip install dask pandas numpy matplotlib
```

```
python dask_project.py
```